

Руководство пользователя экспериментального стенда каскадной архитектуры

Подробная инструкция по установке, запуску, интерпретации результатов и расширению экспериментального стенда для сравнения каскадной интеграции разрешения кореференции в построение семантического графа.

Для кого этот документ:

Для тех, кто будет запускать эксперименты на стенде, читать и интерпретировать его результаты, а также для тех, кто будет расширять стенд новыми резолверами кореференции, бэкендами построения графа или языками. Архитектурный обзор «с высоты птичьего полёта» — в отдельном документе `experimental-stand-scheme.pdf`.

Оглавление

1. Назначение и ключевая идея стенда
2. Установка и требования
3. Структура репозитория
4. Пошаговый запуск первого эксперимента
5. Формат входного корпуса (CoNLL-U / CorefUD)
6. Параметры командной строки
7. Резолверы кореференции и бэкенды графа
8. Формат и чтение результатов
9. Метрики: что они измеряют и как их читать
10. Тестирование стенда
11. Известные ограничения
12. Расширение стенда: новый резолвер или бэкэнд
13. Частые вопросы и типичные ошибки

1. Назначение и ключевая идея стенда

Экспериментальный стенд автоматизирует один прогон исследовательского эксперимента по сравнению **каскадной архитектуры** интеграции разрешения кореференции в построение семантического графа. На входе — корпус с золотой разметкой кореференции в формате CoNLL-U (нотация CorefUD), на выходе — количественное сравнение качества разрешения кореференции и качества построенного из текста графа.

Ключевой инструмент стенда — **oracle-абляция**. Для каждого документа стенд строит граф дважды:

- **oracle-граф** — по тексту, в котором все кореферентные упоминания заменены на форму первого упоминания кластера *с использованием золотой разметки*, без участия какой-либо модели. Это верхняя граница качества построения графа — то, что получилось бы при идеальном разрешении кореференции;
- **predicted-граф** — по тексту, обработанному реальным резолвером кореференции (правило-ориентированным, нейросетевым или LLM).

Разрыв между метриками на oracle-графе и predicted-графе — это оценка «бюджета ошибок», вносимого несовершенством этапа разрешения кореференции в каскадной архитектуре. Именно вокруг этой величины строится гипотеза H1 диссертационного плана: совместная (joint) архитектура должна снижать этот разрыв по сравнению с каскадной — но сравнение с joint-архитектурой является предметом следующего этапа работы, не этой версии стенда.

2. Установка и требования

- 1 **Убедитесь, что рядом с репозиторием `coreference` есть репозиторий `text-corpus-processing`** (в соседнем каталоге, то есть на одном уровне: `../text-corpus-processing` относительно корня `coreference`). Стенд переиспользует его код резолверов и бэкендов графа напрямую через `sys.path`, без копирования. Если репозиторий лежит в другом месте — задайте переменную окружения `TEXT_CORPUSES_PROCESSING_DAGS`, указав в ней путь непосредственно к подкаталогу `dags/` того репозитория.

- 2 **Установите зависимости самого стенда:**

```
pip install -r requirements.txt
```

Это установит `networkx`, `numpy`, `scipy`, `smatch` и `coval`. Обратите внимание: `coval` указан в `requirements.txt` как `coval @ git+https://github.com/ns-moosavi/coval.git` — это осознанное решение: пакет с именем `coval` на PyPI — это **другой, не относящийся к делу проект** (обнаружено и проверено вручную при разработке стенда). Устанавливать нужно именно из указанного git-репозитория.

- 3 **Для реальных прогонов** (не только тестов) установите зависимости `text-corpus-processing`, необходимые конкретным резолверам/бэкендам, которые вы планируете использовать — см. таблицы в разделе 7 и `CLAUDE.md` того репозитория. Для `LapinLiass` нужна модель spaCy `en_core_web_sm`; для `RuleBased` — `en_core_web_lg` и данные WordNet (NLTK); для `SpacyNeural` — модель `en_coreference_web_trf` (требует Python 3.8–3.10 и spaCy 3.4–3.5, см. ограничения версий в исходном репозитории); для `LLMv2` — `torch`, `transformers`, `sentence-transformers` и фактическая загрузка локальной LLM (по умолчанию `Qwen/Qwen2-1.5B-Instruct`) при первом запуске.

Юнит-тесты стенда ничего из этого не требуют. Все обращения к `text-corpus-processing` в тестах заменены на поддельные модули через `sys.modules`, поэтому `pytest` можно запускать сразу после `pip install -r requirements.txt`, даже без второго репозитория рядом.

3. Структура репозитория

```
coreference/
  pipeline/
    types.py          # общие структуры: Token, MentionSpan, CorefDocument, ResolverOutput
    sys_path_setup.py # подключение text-corporates-processing через sys.path
  corpus/
    corefud_loader.py # разбор CoNLL-U с нотацией Entity=
    oracle.py         # построение oracle-текста по золотым кластерам
  resolvers/
    base.py           # интерфейс ResolverAdapter, проекция спанов, UnionFind
    lapin_liass_adapter.py # правило-ориентированный резолвер (EN)
    spacy_neural_adapter.py # нейросетевой резолвер spaCy (EN)
    llm_v2_adapter.py  # LLM-резолвер (любой язык с промпт-файлом)
  graph/
    base.py           # интерфейс GraphBackendAdapter
    rule_based_graph_adapter.py # правило-ориентированный бэкенд (EN)
    llm_v2_graph_adapter.py  # LLM-бэкенд (любой язык с промпт-файлом)
    penman_convert.py  # конвертация графа в AMR-строку для smatch
  eval/
    coref_scoring.py  # CoNLL F1 (MUC/B3/CEAFe) через coval
    graph_scoring.py  # P/R по узлам/рёбрам, доля дублей, Smatch-балл
    report.py         # сборка results.json и report.html
    run_experiment.py # оркестрация прогона + точка входа CLI
  data/corpora/
    sample_en_mini.conllu # маленький пример корпуса для проверки
  tests/pipeline/      # 79 тестов на момент последнего ревью
  docs/superpowers/
    specs/             # дизайн-документ
    plans/             # план реализации по задачам
  experimental-stand-scheme.pdf # архитектурная схема
  manual-ru.pdf        # этот документ
  README.md
```

4. Пошаговый запуск первого эксперимента

1 Установите зависимости стенда и (если планируете реальный прогон, а не только тесты) — зависимости `text-corpus-es-processing` для резолвера `LapinLiass` и бэкенда `RuleBased` (см. раздел 2, шаг 3).

2 Запустите стенд на входящем в репозиторий тестовом корпусе:

```
python -m pipeline.run_experiment \  
  --corpus data/corpora/sample_en_mini.conllu \  
  --language en \  
  --resolvers LapinLiass \  
  --graph-backends RuleBased \  
  --output reports/run1
```

3 После завершения в консоли появится строка `Results written to reports/run1/results.json`, а в каталоге `reports/run1/` — два файла:

- `results.json` — полные метрики по каждой паре «резолвер × бэкэнд» и по каждому документу отдельно (машиночитаемый формат, удобен для дальнейшей агрегации по нескольким прогонам);
- `report.html` — сводная таблица для быстрого визуального просмотра в браузере.

4 Чтобы сравнить несколько резолверов и/или бэкендов в одном прогоне, перечислите их через запятую — стенд автоматически отфильтрует комбинации, не подходящие по языку, и прогонит все оставшиеся:

```
python -m pipeline.run_experiment \  
  --corpus data/corpora/sample_en_mini.conllu \  
  --language en \  
  --resolvers LapinLiass,SpacyNeural \  
  --graph-backends RuleBased \  
  --output reports/run2
```

Для собственного корпуса замените `--corpus` на путь к своему файлу или каталогу `.conllu` и `--language` — на соответствующий код языка. Требования к формату корпуса — раздел 5.

5. Формат входного корпуса (CoNLL-U / CorefUD)

Стенд принимает файлы `.conllu` с кореференцией, размеченной в нотации **CorefUD** — атрибут `Entity=` в десятой колонке (MISC). Поддерживаются:

- одноктоенные упоминания в сжатой записи: `Entity=(e1-person-1-new-1-sgl-1)` ;
- многотокенные упоминания через отдельные открывающую/закрывающую скобки на первом и последнем токене спана: `Entity=(e2-org-...` на первом токене, `Entity=e2)` на последнем;
- несколько сущностей, объединённых в одном значении `Entity=` без разделителя — например, токен, который одновременно закрывает одну сущность и открывает другую: `Entity=e1)(e2-loc-...` . Это — реальная нотация CorefUD (сущности следуют друг за другом *слитно*, не через `|`), и именно на этом ошибалась более ранняя версия загрузчика стенда, пока не была исправлена и покрыта тестами;
- пустые узлы CoNLL-U (идентификаторы вида `N.1` , `N.2`) для нулевой анафоры (zero-anaphora);
- несколько документов в одном файле через комментарии `# newdoc` (с `id = ...` или без — в последнем случае документу присваивается автоматический идентификатор `doc1` , `doc2` , ...).

Восстановление исходного текста

Текст документа восстанавливается из токенов с учётом атрибута `SpaceAfter=No` в MISC (если он есть у токена — следующий токен приклеивается без пробела). Пустые узлы вклада в текст не дают (нулевая ширина), но участвуют в кластерах — упоминание на пустом узле помечается как `is_zero=True` и не подставляется в oracle-текст (там физически нечего подставлять).

Пример минимального документа

```
# newdoc id = sample1
# sent_id = sample1-1
1   John   John   PROP  _      _      2      nsubj  _      Entity=(e1-person-1-new-1-sgl-1)
2   arrived arrive VERB  _      _      0      root   _      SpaceAfter=No
3   .      .      PUNCT _      _      2      punct  _      _

# sent_id = sample1-2
1   He     he     PRON  _      _      2      nsubj  _      Entity=(e1)
2   left   leave VERB  _      _      0      root   _      SpaceAfter=No
3   .      .      PUNCT _      _      2      punct  _      _
```

Полный рабочий пример — файл `data/corpora/sample_en_mini.conllu` в репозитории.

Не поддерживается: многословные токены (multiword tokens, строки с ID вида `3-4` , например для французских стяжений) — их поверхностная форма отбрасывается при восстановлении текста, что может слегка исказить результат на языках, где такие токены часты; разрывные (discontinuous) упоминания с маркерами вида `e1[1/2]` — символы `[` и `]` интерпретируются как часть идентификатора сущности, а не как метаданные разрыва, что может привести либо к разбиению одного кластера на несвязанные псевдо-сущности, либо к ошибке разбора. Подробнее — раздел 11.

6. Параметры командной строки

Параметр	Обязателен	Описание
<code>--corpus PATH</code>	да	Путь к файлу <code>.conllu</code> или каталогу с файлами <code>.conllu</code>
<code>--language CODE</code>	да	Код языка корпуса (<code>en</code> , <code>ru</code> , ...) — фильтрует резолверы/бэкенды по <code>language_support</code> и передаётся в LLM-компоненты
<code>--resolvers NAMES</code>	да	Резолверы через запятую: <code>LapinLiass</code> , <code>SpacyNeural</code> , <code>LLMv2</code>
<code>--graph-backends NAMES</code>	да	Бэкенды через запятую: <code>RuleBased</code> , <code>LLMv2</code>
<code>--output PATH</code>	да	Каталог для <code>results.json</code> и <code>report.html</code> (создаётся автоматически)
<code>--llm-model NAME</code>	нет	Имя модели HuggingFace для <code>LLMv2</code> ; по умолчанию <code>Qwen/Qwen2-1.5B-Instruct</code> . Не влияет ни на что, если <code>LLMv2</code> не запрошен ни как резолвер, ни как бэкенд

Если после фильтрации по языку не осталось ни одной допустимой пары «резолвер × бэкенд», стенд завершается с ошибкой (`RuntimeError` с указанием языка и того, что было отфильтровано) и **не** пишет пустой отчёт. Например, `--language ru --resolvers LapinLiass` всегда завершится ошибкой, так как `LapinLiass` поддерживает только английский — это осознанная защита от тихого «успешного» прогона, который на самом деле ничего не посчитал.

7. Резолверы кореференции и бэкенды графа

Резолверы

Имя	Языки	Источник	Кластеры для CoNLL F1
LapinLiass	только en	правило-ориентированный (spaCy <code>en_core_web_sm</code> + salience-эвристика)	да
SpacyNeural	только en	нейросетевой (spaCy <code>en_coreference_web_trf</code>)	да
LLMv2	любой язык с <code>prompts/coreference_<lang>.txt</code> (сейчас en, ru)	локальная LLM, переписывающая текст	нет

Резолвер `LLMv2` переписывает текст (заменяет местоимения и указательные слова на полные формы упоминаемых сущностей), но не сообщает, какой фрагмент какое упоминание заменил — у него структурно нет выходных «спанов». Для него считается только графовая (oracle-абляционная) метрика; в отчёте его `coreference_metrics` — `null` («N/A» в HTML-таблице). Это архитектурная особенность, зафиксированная в дизайн-документе, а не недоработка.

Бэкенды построения графа

Имя	Языки	Источник
RuleBased	только en	синтаксический разбор spaCy (<code>en_core_web_lg</code>) + лемматизация WordNet
LLMv2	любой язык с <code>prompts/extraction_<lang>.txt</code> (сейчас en, ru)	локальная LLM: извлечение троек → нормализация → дедупликация → сборка графа

В одном прогоне для сравнения oracle- и predicted-графов всегда используется **один и тот же экземпляр бэкенда** — иначе сравнение потеряло бы смысл (разница объяснялась бы разными бэкендами, а не качеством резолвера).

8. Формат и чтение результатов

results.json

Один объект на каждую пару «резолвер × бэкэнд», внутри — усреднённые по документам метрики и полный список метрик по каждому документу отдельно:

```
{
  "run_id": "run1",
  "language": "en",
  "generated_at": "2026-08-04T12:00:00+00:00",
  "results": [
    {
      "resolver": "LapinLiass",
      "graph_backend": "RuleBased",
      "documents": [ /* метрики по каждому документу */ ],
      "coreference_metrics": { "conll_f1": 0.83 },
      "graph_metrics": {
        "node_precision_recall_f1": { "f1": 0.9 },
        "edge_precision_recall_f1": { "f1": 0.75 },
        "smatch": { "f1": 0.81 },
        "oracle_node_duplication_rate": 0.0,
        "predicted_node_duplication_rate": 0.12
      }
    }
  ]
}
```

Внутри каждого элемента списка `documents` хранятся полные, неусреднённые метрики: отдельно precision/recall/f1 для MUC, B³, CEAF_e (не только итоговый CoNLL F1), и все компоненты графовых метрик — это удобно, если нужно посчитать собственную статистику по документам (например, дисперсию между документами), а не только сводку по всему корпусу.

report.html

Таблица со столбцами: резолвер, бэкэнд, CoNLL F1, Node F1, Edge F1, Smatch F1, доля дублей узлов (oracle), доля дублей узлов (predicted). Значение «N/A» — там, где метрика не определена для этой пары (в первую очередь — кореференционные метрики для резолвера `LLMv2`). Откройте файл прямо в браузере.

9. Метрики: что они измеряют и как их читать

Кореференционные метрики (когда у резолвера есть кластеры)

Метрика	Что измеряет
MUC	Долю правильно восстановленных связей между упоминаниями (учитывает кластер как «цепочку» связей, а не как множество)
B³	Пересечение предсказанных и золотых кластеров на уровне отдельных упоминаний, усреднённое по упоминаниям
CEAF_e	Оптимальное (венгерский алгоритм) сопоставление предсказанных кластеров золотым целиком, а не по отдельным упоминаниям
CoNLL F1	Среднее по MUC, B ³ и CEAF _e — стандартная сводная метрика в литературе по кореференции, именно её стоит цитировать при сравнении с чужими результатами

Золотые кластеры-синглтоны (кластер из одного упоминания — по сути, просто размеченное упоминание без пары) отбрасываются перед подсчётом всех трёх метрик — это стандартная практика оценки CoNLL-2012. Если этого не делать, резолвер, который структурно не может выдать синглтон-кластер (а ни один из резолверов стенда этого не делает), получал бы систематически заниженную оценку — не из-за реальных ошибок, а из-за самой процедуры подсчёта.

Графовые метрики (oracle-граф как эталон, predicted-граф как система)

Метрика	Что измеряет
Node P/R/F1	Насколько множество меток узлов predicted-графа совпадает с множеством меток узлов oracle-графа
Edge P/R/F1	Насколько множество направленных троек (источник, метка, приёмник) совпадает; направление ребра значимо — «А сделал В» и «В сделал А» с одной и той же меткой отношения <i>не</i> засчитываются как совпадение
Доля дублей узлов	Доля узлов графа, чья метка совпадает с меткой другого узла <i>этого же</i> графа (внутренняя метрика одного графа, не сравнение двух графов). Разница между значением на predicted- и oracle-графе — это дублирование узлов, вызванное именно ошибками разрешения кореференции (несведённые «Обама» и «Барак Обама» в два разных узла)
Smatch-балл	Структурное сопоставление узлов и рёбер целиком по алгоритму из области разбора AMR (поиск наилучшего соответствия вершин, максимизирующего число совпавших троек) — устойчивее к небольшим отличиям формулировок узлов, чем точное совпадение по меткам

Как читать разрыв oracle vs. predicted: чем ближе метрика на predicted-графе к метрике на oracle-графе того же документа и того же бэнда — тем меньше ошибки разрешения кореференции портят итоговый граф. Этот разрыв — количественная основа гипотезы H1 (совместная архитектура должна сокращать разрыв по сравнению с каскадной), поэтому при отчёте результатов эксперимента имеет смысл показывать оба значения (oracle и predicted) рядом, а не только predicted.

10. Тестирование стенда

```
pytest          # все тесты; внешние модели не требуются — только замканные зависимости
pytest -v       # подробный вывод по каждому тесту
pytest tests/pipeline/test_corefud_loader.py -v        # один файл
```

На момент последнего сквозного ревью реализации: **79 тестов, 0 падений**. Все обращения к тяжёлым внешним зависимостям (spaCy-модели, локальные LLM, код `text-corpuses-processing`) в тестах заменены на поддельные модули через `sys.modules` — тесты можно запускать в чистом окружении сразу после установки `requirements.txt`.

11. Известные ограничения

Зафиксированы по итогам финального сквозного ревью реализации. Не блокируют использование стенда, но должны учитываться при интерпретации результатов.

Ограничение	Пояснение
Разрывные упоминания CorefUD (<code>e1[1/2]</code>)	Не поддерживаются загрузчиком: символы <code>[/]</code> трактуются как часть идентификатора сущности. Встречаются редко; при работе с незнакомым корпусом стоит проверить его на такие маркеры заранее.
Доля дублей узлов для <code>RuleBased</code> всегда 0.0	Честное свойство бэкенда: <code>graphBuilder.merge_graph</code> в <code>text-corpuses-processing</code> хранит узлы как <code>set</code> , дубли меток невозможны уже на уровне построения графа. Метрика содержательна для <code>LLMv2</code> , где узлы различаются по <code>id</code> , а не только по метке.
Пустой корпус → деление на ноль	Файл без строк-токенов при усреднении графовых метрик вызовет исключение. На реальных корпусах с текстом не встречается.
Совместная (joint) архитектура и далее по плану	Мультиязычное пакетное сравнение, статистическая значимость, кривые обучения (H3), анализ ошибок по типам анафоры, нисходящая QA-задача — вне рамок этой версии стенда, см. design-документ, раздел «Out of scope».

12. Расширение стенда: новый резолвер или бэкэнд

Новый резолвер кореференции

Реализуйте класс с интерфейсом `pipeline.resolvers.base.ResolverAdapter` :

```
class MyResolverAdapter:
    name = "MyResolver"
    language_support = {"en", "ru"} # или "any"

    def resolve(self, doc: CorefDocument) -> ResolverOutput:
        ...
        return ResolverOutput(resolved_text=..., clusters=... # or None)
```

Кластеры (`MentionSpan`) должны быть в координатах **золотой токенизации** (`doc.tokens`), а не собственной токенизации резолвера — для проекции используйте `project_char_span_to_gold_tokens` из `pipeline.resolvers.base`. Импорт стороннего кода делайте внутри `resolve()`, а не на уровне модуля — это позволяет юнит-тестам подменять зависимости через `sys.modules` без установки тяжёлых моделей.

Новый бэкэнд построения графа

Интерфейс `pipeline.graph.base.GraphBackendAdapter` :

```
class MyGraphAdapter:
    name = "MyBackend"
    backend_name = "MyBackend"
    language_support = "any"

    def build(self, text: str) -> dict:
        ...
        return {"nodes": [...], "edges": [...]}
```

В обоих случаях после добавления класса зарегистрируйте его имя в `_build_resolver` / `_build_graph_backend` в `pipeline/run_experiment.py`, чтобы имя стало доступно через флаги `--resolvers` / `--graph-backends`. Не забудьте покрыть новый адаптер тестами по тому же шаблону, что и существующие (поддельный модуль стороннего кода через `sys.modules`, проверка на реальном hand-built входе, а не только «не упало»).

13. Частые вопросы и типичные ошибки

«coreference_metrics»: null для LLMv2 — это баг?

Нет. Резолвер LLMv2 не выдаёт спаны упоминаний, только переписанный текст — см. раздел 7. Считается только графовая метрика.

Прогон завершился с RuntimeError про язык

Ни одна из запрошенных пар «резолвер × бэкэнд» не поддерживает указанный `--language`. Проверьте таблицы поддержки языков в разделе 7 — например, `LapinLiass`, `SpacyNeural` и `RuleBased` работают только с `en`.

Первый прогон с LLMv2 «завис»

Не завис — при первом использовании `LLMv2` (резолвер или бэкэнд) локальная LLM (по умолчанию `Qwen/Qwen2-1.5B-Instruct`) скачивается и загружается в память/на GPU. Это может занять продолжительное время в зависимости от скорости интернета и оборудования; последующие запуски в том же окружении используют кэш HuggingFace и стартуют быстрее.

Как посчитать эксперимент на собственном (не английском/русском) языке

Для резолвера/бэкэнда `LLMv2` — добавьте промпт-файлы `prompts/coreference_<lang>.txt` и `prompts/extraction_<lang>.txt` в `text-corpus-processing/dags/llm_v2/` (без изменения кода самого стенда).

Для `LapinLiass` / `SpacyNeural` / `RuleBased` потребуется новый резолвер/бэкэнд — см. раздел 12.

Тесты падают из-за отсутствия spaCy/torch

Юнит-тесты стенда не должны требовать этих зависимостей — все обращения к `text-corpus-processing` в тестах замокированы. Если тест падает именно из-за отсутствия модели — вероятно, это не юнит-тест стенда, а попытка реального прогона `python -m pipeline.run_experiment`; для реальных прогонов зависимости нужны по настоящему, см. раздел 2.

Документ подготовлен по результатам реализации экспериментального стенда (15 задач плана + финальный сквозной ревью, 79/79 тестов). См. также: `README.md`, `experimental-stand-scheme.pdf`, `docs/superpowers/specs/`, `docs/superpowers/plans/`.